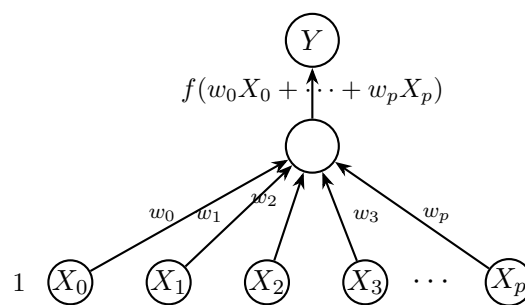


W04-L01

August 10, 2026

Single Layer Perceptron



Input layer

Single layer perceptron

Input-output relationship

$$Y = f\left(w_0 + \sum_{i=1}^p w_i X_i\right)$$

If $f(t) = t$, then we get the linear model.

Usually, f is known and it is an increasing function / non-decreasing function.

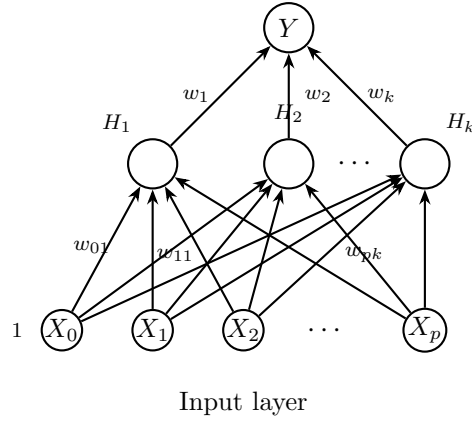
$$\text{example : } \begin{cases} f(t) = \frac{1}{1 + e^{-t}} & \text{Sigmoid} \\ = \max\{0, t\} & \text{ReLU} \end{cases}$$

In general, this gives us a model which can be viewed as a generalized linear model.

Multilayer Perceptron Model

(with one hidden layer)

In order to capture the complex relationship between input and output variables, sometimes we use a complex model, where one or more hidden layers are introduced between the input and the output layers.



Multilayer perceptron model (with one hidden layer)

$$H_j = f_j \left(w_{0j} + \sum_{i=1}^p w_{ij} X_i \right), \quad j = 1, 2, \dots, k$$

$$Y = f \left(w_0 + \sum_{j=1}^k w_j H_j \right)$$

In general,

$$Y = f \left\{ w_0 + \sum_{j=1}^k w_j f_j \left(w_{0j} + \sum_{i=1}^p w_{ij} X_i \right) \right\}$$

If $f(t) = t$,

$$Y = w_0 + \sum_{j=1}^k w_j f_j(\alpha'_j X)$$

$$\alpha_j = (w_{0j}, w_{1j}, \dots, w_{pj}), \quad X = (X_0, X_1, \dots, X_p)$$

i.e. equivalent to write

$$Y = \sum_{j=1}^k f_j^*(\alpha'_j X)$$

which is the projection pursuit model.

Using a multi-layer perceptron model with sigmoid transformations at the hidden layer, one can approximate any continuous function defined on a compact domain to any desired level of accuracy.

$$\sup_x |f(x) - f_{\text{MLP}}(x)| < \varepsilon \quad \text{for any given } \varepsilon > 0$$

provided there are enough neurons in the hidden layer (Hornik, Stinchcombe & White (1989)).

Projection Pursuit Model

Here we are looking for a model of the form

$$f(x_1, \dots, x_p) = \sum_{j=1}^k f_j(\alpha'_j X)$$

and we need to estimate the f_j 's and the α_j 's.

First consider the case $k = 1$.

$$\begin{array}{cccccc} y_1 & x_{11} & x_{21} & \cdots & x_{p1} \\ y_2 & x_{12} & x_{22} & \cdots & x_{p2} \\ \vdots & \vdots & \vdots & & \vdots \\ y_n & x_{1n} & x_{2n} & \cdots & x_{pn} \end{array}$$

How we want to find α_1 and f_1 :

$$\sum_{i=1}^n (y_i - f_1(\alpha'_1 x_i))^2$$

gets minimized.

We can use an iterative method for this purpose.

Note that given any α_1 , we can compute

$$z_1 = \alpha'_1 x_1, \dots, z_n = \alpha'_1 x_n$$

and fit f_1 (approximate f_1) using splines.

How to update α_1 ?

Let $\alpha_1^{(0)}$ be the initial choice of α_1 and $f_1^{(0)}$ be the current estimate of f_1 .

$$\begin{aligned} \sum [y_i - f_1(\alpha'_1 x_i)]^2 &\approx \sum [y_i - f_1^{(0)}(\alpha'_1 x_i)]^2 \\ &\approx \sum \left[y_i - f_1^{(0)}(\alpha_1^{(0)'} x_i) - (\alpha_1 - \alpha_1^{(0)})' \nabla f_1^{(0)}(\alpha_1^{(0)'} x_i) \Big|_{\alpha_1 = \alpha_1^{(0)}} \right]^2 \\ &= \sum [y_i^* - (\alpha_1 - \alpha_1^{(0)})' \delta_i^*]^2 \end{aligned}$$

$$\begin{aligned} \Psi(\alpha) &\approx \Psi(\alpha_0) + (\alpha - \alpha_0)' \Psi'(\alpha) \Big|_{\alpha = \alpha_0} \\ \Psi(\alpha) &= \Psi(\alpha_0) + (\alpha - \alpha_0)' \nabla \Psi(\alpha) \Big|_{\alpha = \alpha_0} \end{aligned}$$

We can find the minimizer and that can be used as $\alpha_1^{(1)}$.

We can proceed this way to get the final estimates of α_1 and f_1 .

If $K > 1$ (e.g. assume that $K = 2$):

After getting α_1 and f_1 , we redefine the response variable

$$\tilde{y} = y - f_1(\alpha'_1 X)$$

and proceed similarly.

$$\sum (y_i - f_1(\alpha'_1 x_i) - f_2(\alpha'_2 x_i))^2$$

$$\sum (\tilde{y}_i - f_2(\alpha'_2 x_i))^2$$

We stop when the reduction in RSS(Residual sum of squares) is less than a user-defined threshold.

Other Estimates for Regression Function

Nearest Neighbor Method

X	5	7	8	8.5	9.7	10.6	11.6	13
Y	11	13	14	15	14	13	12	10

(Example with $k = 3$, $\hat{y} = 14$)

$$\hat{f}_k(x) = \frac{\sum_{x_i \in N_k(x)} y_i}{k}$$

where $N_k(x)$ is the collection of k -neighbors of x .

This nearest neighbor estimate is piecewise constant.

This can be easily generalized to higher dimension.

In order to get the consistency of k -VN estimator, we need

$$k \rightarrow \infty \quad \text{and} \quad \frac{k}{n} \rightarrow 0 \quad \text{as} \quad n \rightarrow \infty$$

(Stone, 1977).

Weighted Nearest Neighbor Method

Instead of nearest neighbor, one can use the weighted nearest neighbor method. In that case, the estimator is defined as

$$\hat{f}_{k\text{-WNN}}(x) = \frac{\sum_{i: x_i \in N_k(x)} w(\|x - x_i\|) Y_i}{\sum_{i: x_i \in N_k(x)} w(\|x - x_i\|)},$$

where $w : \mathbb{R}_+ \rightarrow \mathbb{R}_+$ is a non-increasing function.

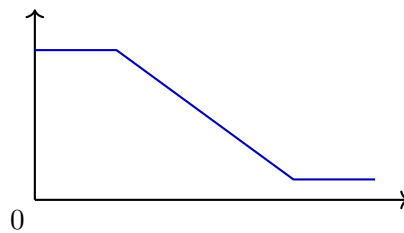


Illustration of a non-increasing weight function w